FAIRLANE · PHASE II REHEARSAL GUIDE · FILE 2 OF 3

Live Demo

Scheduler + worker agent, step by step on both machines

Team Orchestrators · Fairlane: A Fair and Fault-Tolerant Scheduler for Distributed ML Training Workloads

WHO IS WHO

- VANSH'S PC = the central SCHEDULER (FastAPI + PostgreSQL)

```
folder: ~/Code/GPU_Orchestrator
```

- SURYANSH-VM = the WORKER AGENT (runs the jobs)

```
folder:
~/Distributed_GPU_Workload_Orchestrator/g
pu-orchestrator/worker-agent
```

WHAT THE DEMO SHOWS

1. A worker joins the cluster by itself (registration + heartbeats).
2. The scheduler picks jobs fairly (professor before student).
3. A crashing job is reported with its real exit code.
4. A job that is too big for any GPU is never sent anywhere.
5. A job submitted live is picked up within ~2 seconds.

(The worker-crash recovery is in File 3.)

HOW TO READ THIS FILE

- Lines under `>> PASTE:` are commands. Copy them exactly, one line at a time, and press Enter after each.
- Replace **<VANSH_IP>** and **<VM_IP>** with the real numbers (no < >).
- `EXPECT:` tells you what a correct result looks like.
- `SAY:` is the line to tell the evaluator.

TERMINALS YOU NEED

Vansh's PC : Terminal V1 (server stays running) + Terminal V2 (commands) suryansh-vm: Terminal S1 (agent stays running) + Terminal S2 (spare)

TIP make the terminal font big (Ctrl + Shift + +) so the evaluator can read it.

Total Time about 6-8 minutes

PART 0 Pre-flight (Do This Before the Evaluator Arrives)

0.1 Same network

Both laptops (Vansh's PC and Suryansh's Windows PC) on the SAME phone hotspot. Not college Wi-Fi (it blocks laptop-to-laptop traffic).

0.2 Suryansh: VM network is Bridged

VirtualBox → VM → Settings → Network → `Bridged Adapter`, Name = your Wi-Fi adapter. Then inside the VM:

PASTE suryansh-vm S2

```
sudo nmcli networking off && sudo nmcli networking on
```

0.3 Get both IPs

PASTE Both machines

```
hostname -I
```

Use the FIRST address (ignore 172.17.x.x, that is Docker).

VANSH_IP _____ **VM_IP** _____

0.4 Vansh: database and firewall ready

PASTE Vansh's PC V2

```
sudo systemctl start postgresql
sudo ufw status
```

If ufw says `Status: active`:

PASTE Vansh's PC V2

```
sudo ufw allow 8000/tcp
```

0.5 Suryansh: create the demo job scripts

PASTE suryansh-vm S1

```
cd ~/Distributed_GPU_Workload_Orchestrator/gpu-orchestrator/worker-agent
source .venv/bin/activate
bash setup_demo_jobs.sh
```

EXPECT

```
Demo job scripts created under /home/suryansh-bhatnagar/fairlane_jobs/users
```

Your prompt now starts with (.venv). Keep it that way in S1.

PART 1 Fresh Start (Vansh's PC)

1 Reset the database and load the demo data

PASTE Vansh's PC V2

```
cd ~/Code/GPU_Orchestrator
uv run python -m scripts.create_tables --reset
uv run python -m scripts.seed
```

EXPECT
no errors (no output is fine).

SAY TO THE EVALUATOR
"We start with two users: Jack, a student, and Jane, a professor. There are five jobs waiting. One of them, huge-llm, needs 16 GB, more than any GPU we have."

2 Start the scheduler

PASTE Vansh's PC V1

```
cd ~/Code/GPU_Orchestrator
uv run uvicorn fairlane.main:app --host 0.0.0.0 --port 8000
```

EXPECT
`Uvicorn running on http://0.0.0.0:8000`. Leave V1 running.

IMPORTANT --host 0.0.0.0 is required, otherwise the VM cannot connect.

3 WAIT 40 SECONDS, then check status

WHY the demo data contains three pretend workers (gpu-node-1/2/3) that never send heartbeats. After 30 s the scheduler marks them OFFLINE. If you start the agent too early, a few jobs are first sent to these pretend machines and show `dispatch failed: ConnectTimeout` (harmless - they get re-queued - but it looks messy).

PASTE Vansh's PC V2

```
uv run python -m scripts.status
```

EXPECT under WORKERS

```
1 gpu-node-1  OFFLINE
2 gpu-node-2  OFFLINE
3 gpu-node-3  OFFLINE
```

SAY TO THE EVALUATOR
"These three workers stopped sending heartbeats, so the reaper marked them offline. Nothing will be scheduled on them."

PART 2 Connect the Worker (suryansh-vm)

4 Check the VM can reach the scheduler

PASTE suryansh-vm S2

```
curl -s http://<VANSH_IP>:8000/docs | head -3
```

EXPECT

a few lines of HTML (<!DOCTYPE html> ...).

If `curl: command not found`: sudo apt install -y curl then retry. If it hangs or `No route to host`: see TROUBLESHOOTING at the bottom.

5 Start the worker agent

PASTE suryansh-vm S1 one single line

```
SERVER_URL=http://<VANSH_IP>:8000 AGENT_URL=http://<VM_IP>:8001 WORKER_NAME=suryansh-vm bash run_agent.sh
```

EXPECT

```
The first line shows your REAL IPs (not the words VANSH_IP / VM_IP)
... registered with http://<VANSH_IP>:8000 as worker 4 (suryansh-vm)
```

Then, as jobs run, lines like:

```
execution 2: completed (exit 0) -> scheduler HTTP 200
execution 4: failed (exit 3) -> scheduler HTTP 200
```

Leave S1 running.

SAY TO THE EVALUATOR

"When the agent starts, it detects its GPUs, registers itself with the scheduler and sends a heartbeat every few seconds. Nobody configured this worker on the server - it joined on its own."

6 Vansh confirms he can reach the worker

PASTE Vansh's PC V2

```
curl -s http://<VM_IP>:8001/healthz
```

EXPECT

{"status":"ok"}

PART 3 Watch the Jobs Run (Both, About 1-2 Minutes)

7 Watch

Suryansh: watch S1 - jobs start and finish one after another. Vansh: run status every ~10 seconds:

PASTE Vansh's PC V2 — repeat as needed

```
uv run python -m scripts.status
```

Each normal job takes about 20 seconds.

8 Point out these four things on the status screen

a **cifar-b was sent BEFORE mnist-a (look at EXECUTIONS: cifar-b, job 3, has the smaller execution number)**

> **SAY TO THE EVALUATOR**
> *"mnist-a was submitted earlier, but cifar-b belongs to Jane, a professor, so it has higher priority. The scheduler's formula combines role priority, waiting time (aging) and a penalty for users who are already using a lot of GPU."*

b **bench-c → FAILED `exit code 3`**

> **SAY TO THE EVALUATOR**
> *"This script crashes on purpose. The worker reported the real exit code back to the scheduler, so the user sees exactly what happened."*

c **huge-llm → stays QUEUED forever**

> **SAY TO THE EVALUATOR**
> *"It needs 16 GB and our biggest GPU has 12 GB. The scheduler checks memory per GPU and never sends a job that cannot fit, instead of letting it crash."*

d **Everything else → COMPLETED on worker 4**

> **EXPECT** (final)

```
JOBS
  1 huge-llm   16000 MiB  QUEUED
  2 mnist-a     3000 MiB  COMPLETED
  3 cifar-b     6000 MiB  COMPLETED
  4 bench-c     2000 MiB  FAILED
  5 cifar-d     5000 MiB  COMPLETED
```

PART 4 Submit a Job Live (Vansh's PC)

9 Submit a new job

PASTE Vansh's PC V2

```
uv run python -m scripts.submit Jane late-job 4000
```

EXPECT

```
submitted job 6 (late-job, 4000 MiB) for Jane
```

Within ~2 seconds a new job starts in S1.

SAY TO THE EVALUATOR

"The scheduler loop runs every 2 seconds, so new work is picked up almost immediately."

10 After ~20 seconds, show it finished

PASTE Vansh's PC V2

```
uv run python -m scripts.status
```

EXPECT

6 late-job 4000 MiB COMPLETED

PART 5 Show the Logs (suryansh-vm)

11 Open the job logs

PASTE suryansh-vm S2

```
cd ~/Distributed_GPU_Workload_Orchestrator/gpu-orchestrator/worker-agent
ls logs/
tail -n 3 logs/job_exec*.out
cat logs/job_exec*.err
```

EXPECT

two files per execution (.out = normal output, .err = errors). The .out files show `training ... done`; one .err file (bench-c) contains `simulated crash`.

SAY TO THE EVALUATOR

"Every run's output is captured to a log file, and the path is sent back to the scheduler with the result."

NEXT If you are also doing the crash test, go to File 3 NOW (keep both the server and the agent running).

PART 6 Reset to Run the Demo Again

1. Suryansh: in S1 press Ctrl+C (stops the agent; fine here because no job is running).
2. Vansh: in V1 press Ctrl+C (stops the server).
3. Start again from STEP 1.

NOTE Never press Ctrl+C on the agent WHILE a job is running during the demo - it kills that job and it is recorded as FAILED (`exit code -2`). Use File 3 for crash testing.

Troubleshooting

WHAT YOU SEE	WHAT TO DO
Agent error `Temporary failure in name resolution`	You typed the words VANSH_IP / VM_IP. Put the real numbers in Step 5.
Agent error `No route to host` or ping fails	Different networks. Both laptops on the same hotspot; VM on Bridged (Step 0.2).
Ping works, curl to :8000 fails	Server not started with --host 0.0.0.0, or Vansh's firewall: sudo ufw allow 8000/tcp
Agent registered but no jobs arrive	Vansh runs Step 6. If it fails, VM_IP is wrong or VM not Bridged.
Job FAILED `Script ... not found`	Suryansh reruns: bash setup_demo_jobs.sh
`dispatch failed: ConnectTimeout` on worker 1/2/3	Agent started before Step 3's wait. Harmless; those jobs are retried on worker 4. Explain it as `failed dispatch is retried automatically`.
Server error about DATABASE_URL	Run from ~/Code/GPU_Orchestrator (the .env file is there), and PostgreSQL is started.
`No module named scripts.submit`	You ran it on the VM. Scheduler commands (scripts.*) only run on Vansh's PC.
Status still shows worker 4 ONLINE after the jobs end	Normal - the agent is still running.